```
import com.polarbookshop.orderservice.order.domain.OrderService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import reactor.core.publisher.Flux;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OrderFunctions {

  private static final Logger log =
    LoggerFactory.getLogger(OrderFunctions.class);

  @Bean
  public Consumer<Flux<OrderDispatchedMessage>> dispatchOrder(
    OrderService orderService
  ) {
    return flux ->
      orderService.consumeOrderDispatchedEvent(flux)                    ❶
        .doOnNext(order -> log.info("The order with id {} is dispatched",
          order.id()))                                                  ❷
        .subscribe();                                                   ❸
  }
}
```

❶ For each dispatched message, it updates the related order in the database.

❷ For each order updated in the database, it logs a message.

❸ Subscribes to the reactive stream in order to activate it. Without a subscriber, no data flows through the stream.

Order Service is a reactive application, so the `dispatchOrder` function will consume messages as a reactive stream (a `Flux` of `OrderDispatchedMessage`). Reactive streams are *activated* only if there's a subscriber interested in receiving the data. For that reason, it's critical that we end the reactive stream by subscribing to it, or else no data will ever be processed. In previous examples, the subscription part was handled transparently by the framework (for example, when using reactive streams to return data via a REST endpoint or to send data to a backing service). In this case, we have to explicitly do that with the `subscribe()` clause.